

VIDEO ANALYTICS AND MACHINE LEARNING

Group Coursework

Remo Divito, Joshua Young, Ben Irwin

2024

Introduction

This report outlines the development and evaluation of a machine learning-based face detection system as part of our CSC3067 coursework. The coursework focuses on the implementation and testing of classification models and their integration into a comprehensive detection pipeline. The project emphasizes the use of feature extraction methods, classifier models, and parameter optimization to achieve robust and reliable performance.

The task involves training and testing three classification models—KNN, Random Forest, and SVM—using a dataset comprising labelled face and non-face images. A series of experiments are conducted to evaluate the impact of various parameters, preprocessing techniques, and feature extraction methods on classification accuracy and F1 score. These findings inform the selection of the optimal classifier for the face detection system.

To implement the detection system, the sliding window technique and Non-Maximum Suppression (NMS) are employed to identify faces across images at multiple scales while minimizing false positives. The final system integrates the selected classifier and employs parameter-tuned settings to maximize detection accuracy.

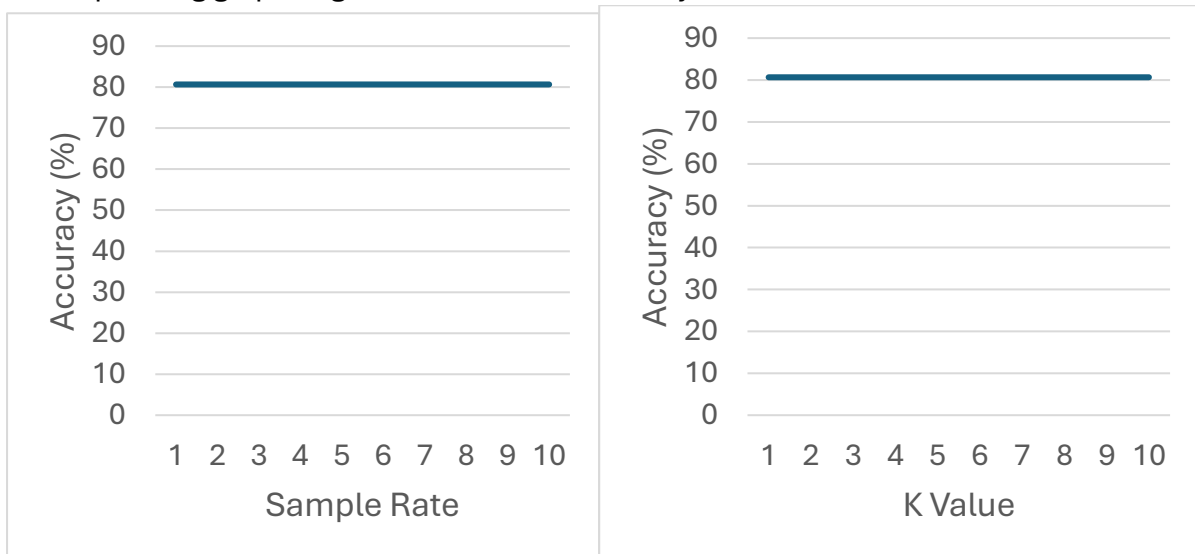
This report details each stage of the project, including classifier comparison, preprocessing, feature extraction, data augmentation, and the sliding window implementation. Emphasis is placed on justifying decisions with quantitative results and aligning system design with the module's objectives.

1. Classifier Model

We implemented and tested 3 classification models using the datasets provided, KNN, Random Forrest, and SVM. We decided to do some testing of these models to decide which is most suitable for our use case, before fine tuning it and implementing it in the next stage of the face detection system.

1.1 KNN

KNN was implemented and tested using varying K values and sample rates, here are the corresponding graphs against the model accuracy.



Here we see a constant accuracy rate of 80.65% regardless of the parameter values used. At a glance this is a respectable accuracy rate for such a simple classifier, however looking deeper, this reflects the exact ratio of the number of face to non-face images in the training dataset, meaning it is very heavily biased in returning a positive result. This is likely due to the high dimensionality of the feature vectors we were using to train and test the model, KNN is known to not work well for high dimension vectors.

Dimension Reduction

An attempt was made for dimension reduction on the feature set used, however this was unfortunately too resource intensive to run on my machine, the code for this dimension reduction is still in the code for FACE_KNN.m, but it is commented out.

Running the Code

To run this code, simply run FACE_KNN in the terminal, and the accuracy will be printed to the terminal.

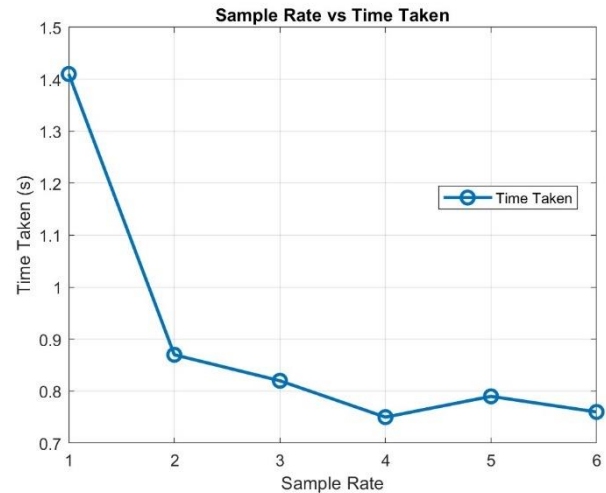
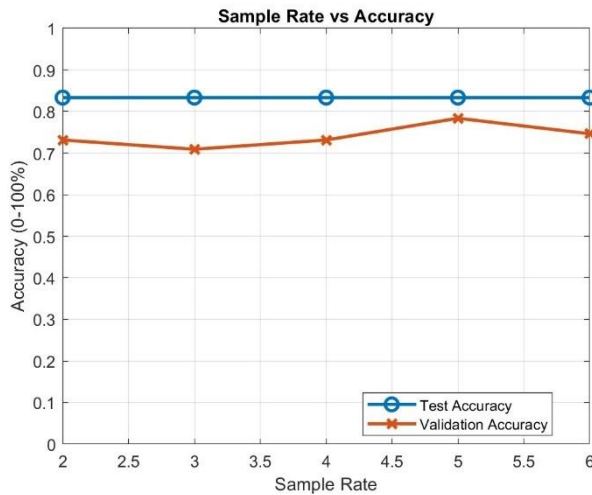
Final Comments

It goes without saying this classification model is unsuitable for our use case due to its lack of accuracy.

1.2 Random Forrest

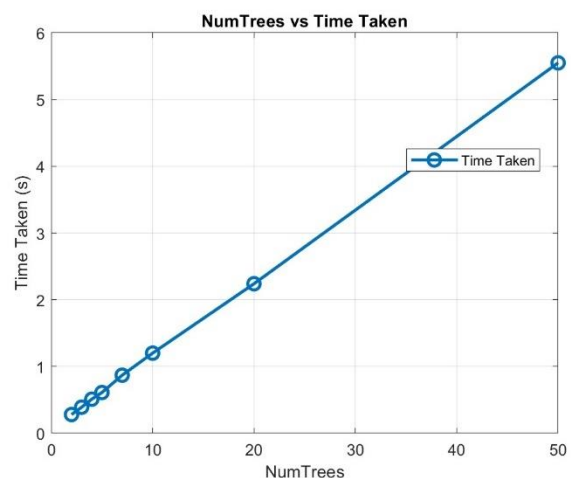
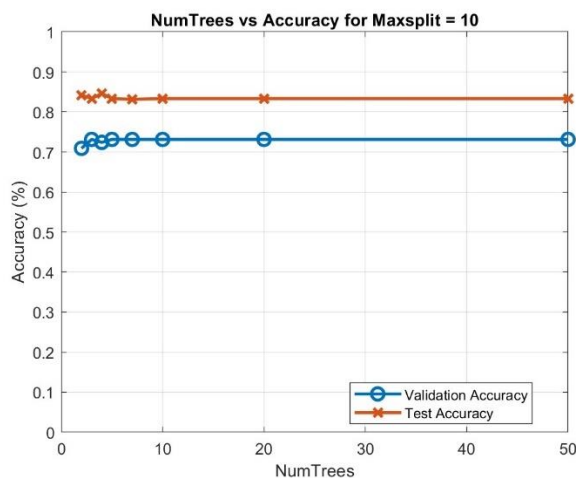
Random Forrest was implemented and tested using varying sample rate, number of trees, and max split values, here are the corresponding graphs against the model accuracy, and time taken.

Sample Rate



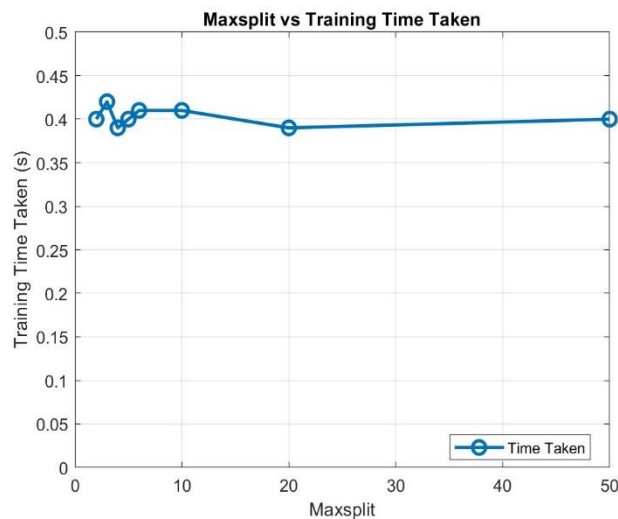
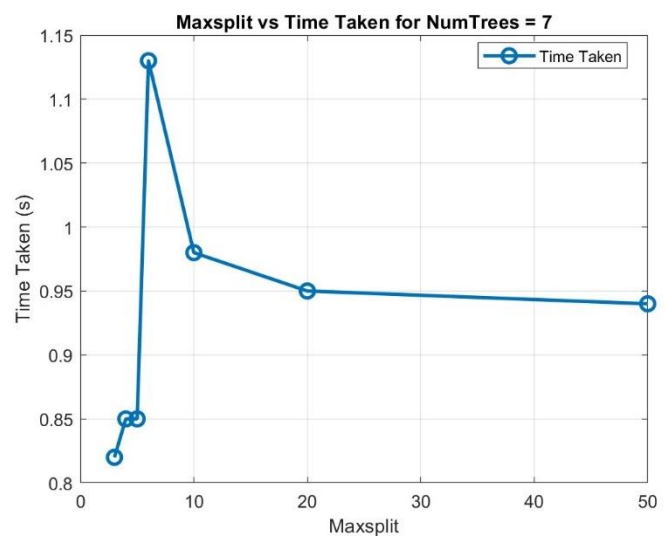
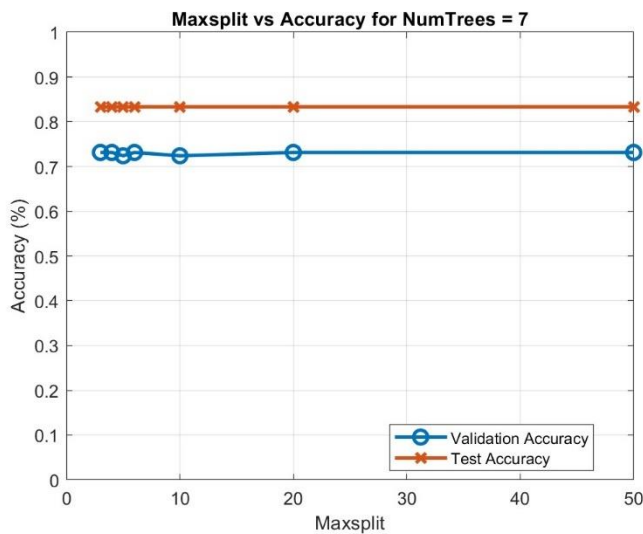
Increasing the sample rate for the training algorithm reduces the time taken to train a model. Unexpectedly the sample rate seems to have no significant effect on test accuracy. My initial assumption was that the smaller data size at higher sample rates would cause the accuracy to drop. The validation accuracy, which is the percentage of faces in the training dataset, changes expectedly as the sample rate and dataset changes.

Number of Trees



As the number of trees in the model increase, the time taken for testing increases linearly. The number of trees has little effect on accuracy, but the values seem to stabilise around 5 trees.

Max Split



The value of Maxsplit seems to have little effect on accuracy for both validation and testing. The graph shows that as Maxsplit increases so does the time taken for testing but not the time taken for training.

Running the Code

To run this code, simply run `train_test_RF_Model` in the terminal, and the accuracy will be printed to the terminal.

Final Comments

Given the testing results for Random Forrest, these are the best parameter values we've found:

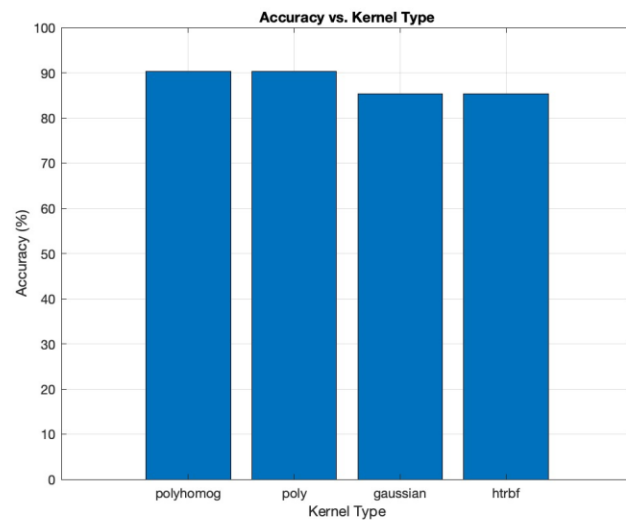
- NumTree = 7 – Accuracy seems to be stable while time taken is still low.
- Maxsplit = 20 – Time taken is low ~ provisional value: could change further testing
- SampleRate = 4 – Time taken is at its lowest while maintaining accuracy

With these parameters Random Forest achieves 83.33% accuracy on the testing dataset. This is respectable, and will be considered for the next stage in the system depending on the outcome of our SVM testing.

1.3 SVM

SVM was implemented and tested using varying kernel types, polynomial degrees, and sample rates, here are the corresponding graphs against the model accuracy.

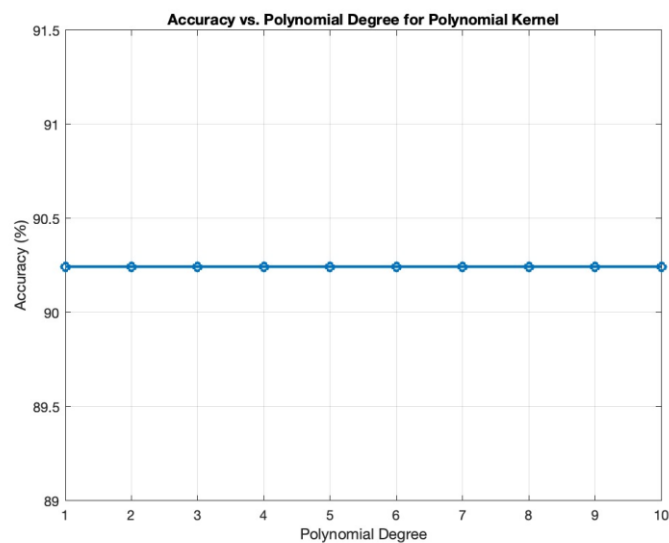
Kernel Type



The model was trained using the polynomial Kernel type, as this was found to be the most accurate (along with the homogenous polynomial kernel)

Each iteration of the model was trained with a 'kerneloption' parameter of 1, or in the case of the htrbf kernel, '[1,1]'

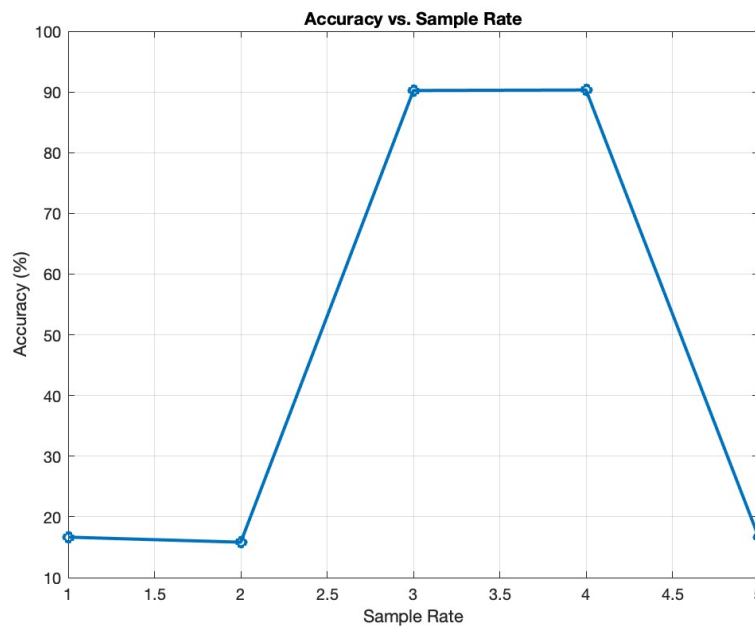
Kernel Option (Polynomial Degree)



The 'kerneloption', parameter of the SVM model, which represents the polynomial degree of the kernel, was set to 1, as it was not found, after testing numbers from 1 to 10, to have any effect on accuracy.

An unchanged accuracy was also returned after testing with different values for the lambda and C parameters – these are statically set to '1e-4' and '1' respectively.

Sample Rate



A sample rate of 4 was used for image selection from the dataset for training

- Sample rates of 3 and 4 were found to have the highest accuracy
- The higher value was chosen to maximise the computation efficiency of training the model

Running the Code

To run this code, simply run FACE_SVM in the terminal, and the accuracy will be printed to the terminal.

Final Comments

Initially when we we're running these tests against the SVM model we were getting an accuracy of 91.59%. This was enough for SVM to be our choice of model going forward. However, it may be important to note that after implementing and testing the final face detection code, we found that setting the prediction threshold for SVM to 0.7 (up from 0) offered the best results. Rerunning the initial tests with this prediction threshold returns a 96.9% Accuracy.

1.4 Conclusion

We have decided to proceed with the SVM classification model because it aligns well with our use case of working with a relatively small dataset. SVM's strength lies in its ability to perform effectively with limited training data, making it an ideal choice given the constraints of our dataset. This decision is empirically supported by our testing results, where SVM achieved a classification accuracy of 96.9%, outperforming Random Forest by 13.6 percentage points and KNN by 16.25 percentage points. KNN also struggles with high-dimensional data and so is unsuitable for this use case. These findings confirm that SVM is the most robust and reliable model for our application.

2. Metrics

Three different splits of the training data were considered. In this context, ‘positive’ refers to images with the label corresponding to a positive identification of a face, and ‘negative’ refers to images that correspond to a negative identification of a face:

First, an uneven split partitioning of training and testing data. Out of 124 images:

- 94 were assigned to the training set
 - o 49 positive, and 45 negative
- 30 were assigned to the testing set
 - o 21 positive, and 9 negative

Then, another fixed partition split nearly equally between the training and testing sets (with a slight bias to the training set to avoid overfitting as much as possible).

- 63 were assigned to the training set
 - o 35 positive, and 28 negative
- 61 were assigned to the testing set
 - o 33 positive, and 29 negative

Finally, cross-validation was used, with 10 folds

When compared in otherwise equal conditions, the following results were obtained:

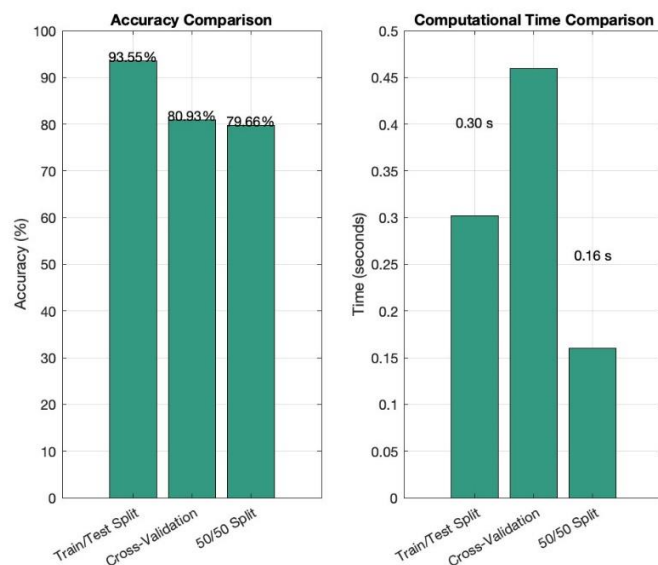


Figure 2.1

Figure 2.1 appears to show a significantly increased accuracy for the original train/test split. However, if the splits are run against F1 score instead of accuracy, a stark difference emerges:

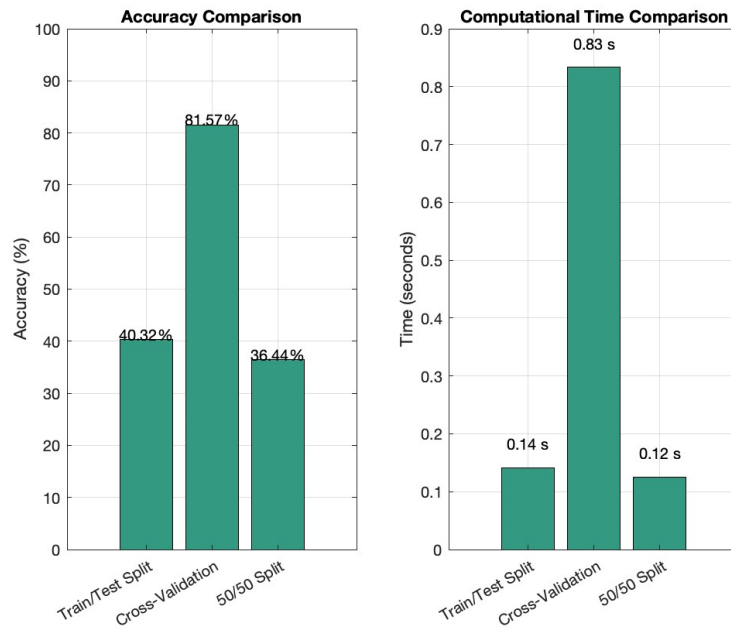


Figure 2.2

Figure 2.2 shows that using a cross-validation split returns a much higher F1 score than the static train/test partitioning and the 50/50 split. The delta between the accuracy and F1 score of the train/test split suggests that while the model performs well with this split overall, the imbalance of positive and negative images in the dataset may have skewed this accuracy.

In order to determine which number of folds provides the highest F1 score, and thus the greatest performance in terms of precision and recall, each number of folds between 2 and 10 were tested against each other with otherwise equal parameters.

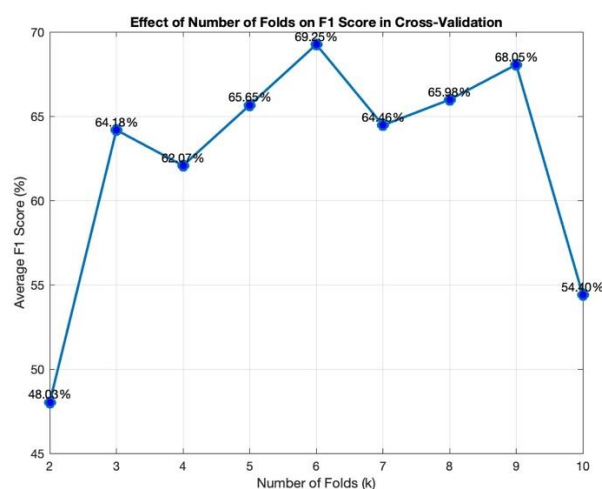


Figure 2.3

Figure 2.3 shows that using 6-fold validation produced the highest F1 score, of 69.25%. After conducting these tests, it was decided to move forward with 6-fold cross-

validation, and the F1 score for comparing different models, methods and parameters.

From this comparison, it can be gathered that cross-validation, despite taking longer to process, is a far more robust and reliable method of splitting the data, likely due to the fact that it rotates the training and validation sets across the whole dataset, is less vulnerable to bias from underrepresented classes.

When using cross-validation, an effort was made to ensure that there was no data leakage, i.e. that augmented images were not spread between the training and testing classes.

It also appears that using the F1 score as a means of comparison, rather than accuracy, maybe more reliable and revealing and less likely to show inflated numbers due to uneven datasets – particularly useful given the small size of the dataset.

3. Preprocessing, Data Augmentation and Sample Rate

Before applying the machine learning model, there are a few considerations and techniques that may be altered to increase the reliability of the model. These include feature extraction (discussed in the next section), preprocessing techniques to enhance the images before feature extraction, data augmentation including flipping and rotating the dataset to artificially increase the size of the dataset, and adjusting the sample rate that images are picked at for use in the model.

3.1 Preprocessing techniques

Noise reduction and histogram equalisation were compared against the unaltered to determine whether these techniques may increase the F1 score.

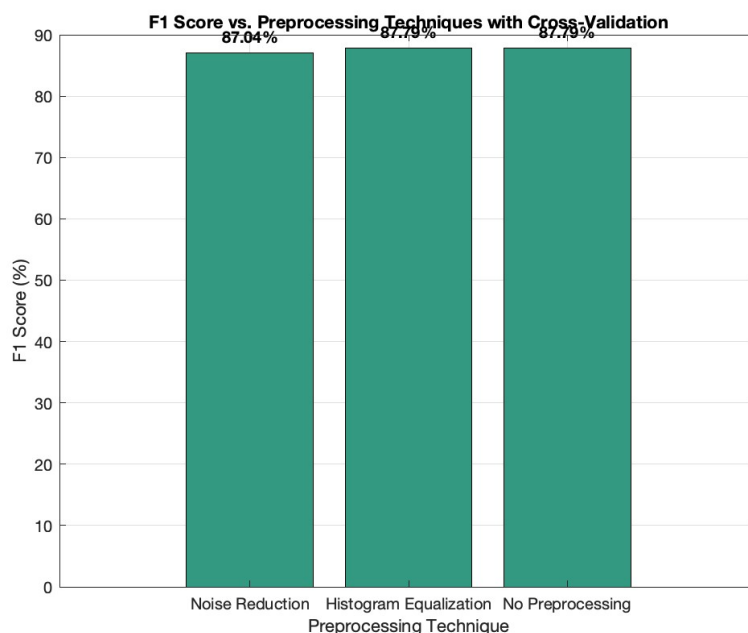


Figure 3.1

Figure 3.1 shows that histogram equalisation made no change on the F1 score when compared to no preprocessing, and applying noise reduction resulted in a slight dip in F1 score, at 87.04% compared to 87.79% with no preprocessing.

This suggests the dataset has no need for these preprocessing techniques and is not made easier to classify because of them.

After reviewing this data, it was decided not to move forward with any preprocessing technique.

3.2 Data Augmentation

The model was tested once with, and once without data augmentation.

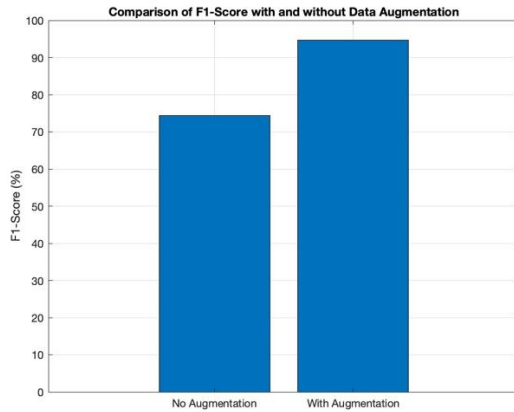


Figure 3.2

Figure 3.2 demonstrates a significant improvement in F1 score from 75% to 95% after applying data augmentation. This sharp increase highlights the effectiveness of augmentation in enhancing the model's ability to generalize to real-world scenarios. By artificially increasing the dataset size through rotations and flips, the augmented images simulate variations in angles and perspectives that are likely to occur in practical applications. This improvement is particularly valuable given the constraints of the relatively small original dataset, as it addresses potential overfitting and ensures the model is exposed to more diverse training examples.

Because of this sharp 20% increase in F1 score, data augmentation is used in the machine learning model.

3.3 Sample Rate

The rate at which images from the dataset were picked and loaded before being applied to the model was varied and compared against F1 score.

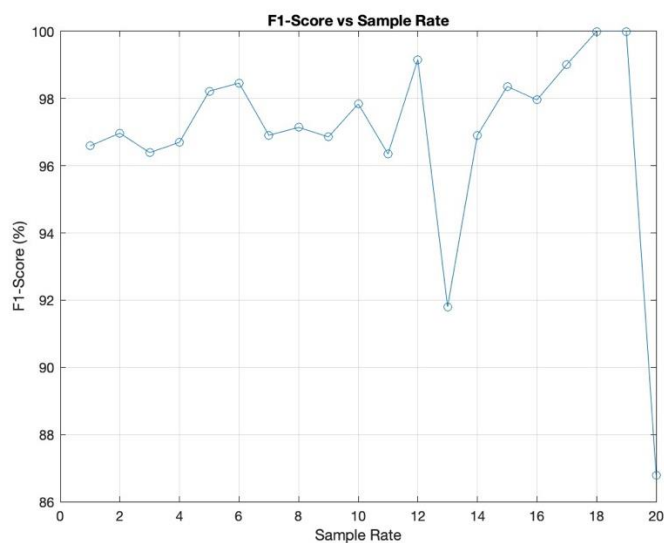


Figure 3.3

Interestingly, Figure 3.3 shows that the highest cross-validated F1 scores, both 100%, appear at sample rates of 18 and 19. These represent the already relatively small datasets being reduced by factors of 18 and 19, respectively, so the reliability of these results is dubious. Despite the higher F1 score gained by higher sample rates, a sample rate of 1 was chosen to maximise the amount of data the model was exposed to, so that it may be more reliable.

4. Feature Extraction

A number of feature extraction techniques were compared to see which would facilitate the highest F1 score. These were Histogram of Gradients, Gabor filter, Scale Invariant Feature Transform, and Raw Pixels (no extractor).

4.1 SIFT Extractor

The SIFT feature extractor identifies 'key points' in an image and determines descriptors for the image, which are converted into a visual Bag-of-Words to lower the dimensionality for use with a machine learning model – SVM in this case.

The open-source VLFeat library was used to facilitate SIFT extraction and implementation via BoW. Source: <https://www.vlfeat.org>.

First, the number of words in the BoW 'vocab' was varied and compared against F1 score.

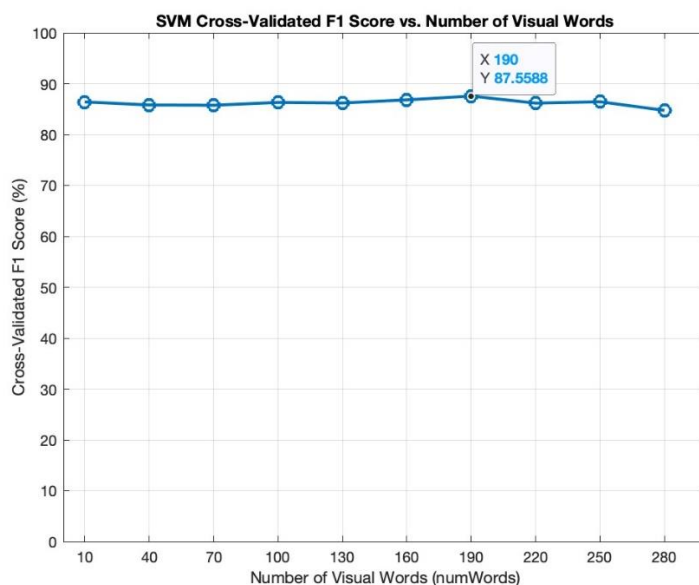


Figure 4.1

Figure 4.1 shows that the highest cross-validated F1 score appears to be at around 190 words. This number of words may return the highest F1 score because the SIFT model may have found a relatively high number of highly distinctive features, suggesting relatively high feature diversity amongst the image dataset,

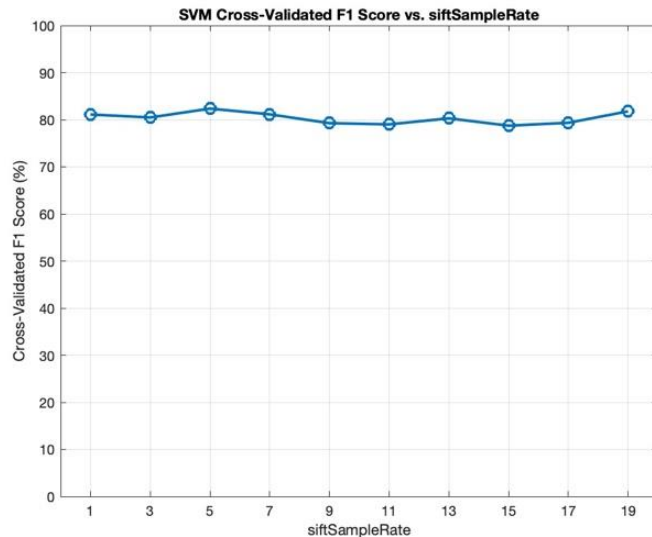


Figure 4.2

Figure 4.2 shows the comparison of different SIFT sample rates, which determine how finely the image is sampled for feature extraction.

The F1 score appears relatively stable but peaks around 5. This relatively high sample rate will lead to fewer features. This implies that anything lower (sample rates of 1 to 5), although producing more features, may lead to overfitting in that it gathers features from unnecessary or irrelevant details. For this reason we have decided to use a sample rate of 5.

4.2 Comparing Feature Extractors

With the SIFT extractor optimised, it was compared against HOG, Gabor Filter, and Raw Pixels.

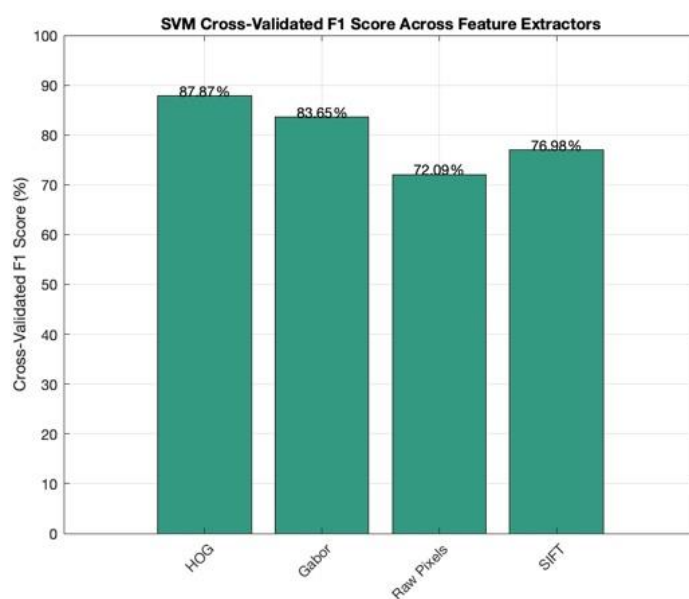


Figure 4.3

Figure 4.3 shows that the Histogram of Gradients technique returned a higher F1 score than the SIFT and Gabor extractors, with scores of 87.87%, 83.65%, and 76.98% relatively. All feature extractors scored higher than the raw pixels with no extractor, which scored 72.09 %.

The higher F1 score implied the HOG feature extractor was the most robust, so it was applied to the dataset in the detector.

5. SVM Kernel

A number of SVM Kernel types were tested: homogenous polynomial, polynomial, gaussian, and high-tailed radial basis function.

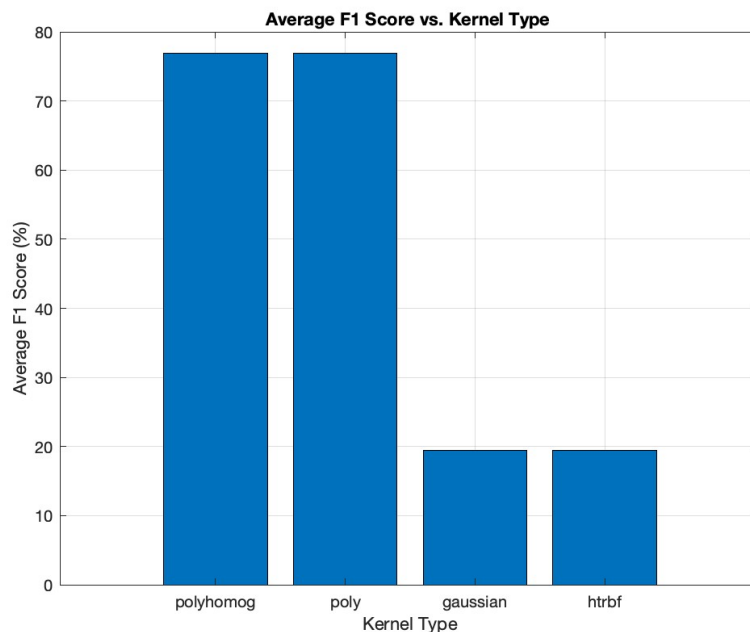


Figure 5.1

Figure 5.1 shows that the average cross-validated F1 score of the ‘poly’ and ‘polyhomog’ kernels were the highest. The ‘kerneloption’ parameter (see below) was set to 1 for the ‘polyhomog’, ‘poly’ and ‘gaussian’ kernels and [1,1] for the ‘htrbf’ kernel as a default for a fair comparison.

The fact that polynomial kernels performed better may suggest that the data is close to linearly separable, which would also mean more complex kernels like the Gaussian and HTRBF could draw overly complex boundaries that lead to overfitting.

Given the F1 score for the polynomial and homogenous polynomial kernels were the same, the homogenous polynomial kernel was used due to its slightly improved computational efficiency as a result of the lack of a bias term (c) when compared to the regular polynomial kernel.

5.1 Kernel Option (Polynomial Degree)

The SVM function supplied in the SVM-KM toolbox allows for a kernel option to be passed as a parameter. For each kernel type this represents something different, but in the case of the polynomial, or in this case homogenous polynomial kernel this represents the polynomial degree.

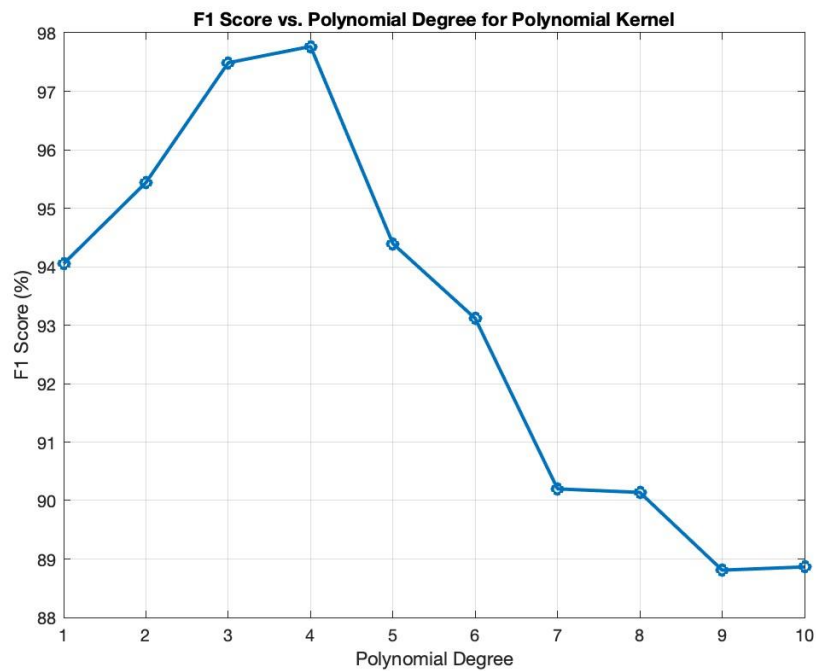


Figure 5.2

The model was found to return the highest F1 score at 4 polynomial degrees, marginally better than 2 and 3 polynomial degrees, so this was used for the SVM model.

6. Dimensionality Reduction

The SVM model at this point was nearly fully complete, with all necessary parameters tuned and set after thorough experimentation. It was decided to investigate principal component analysis, or PCA, to see if it would have a positive effect.

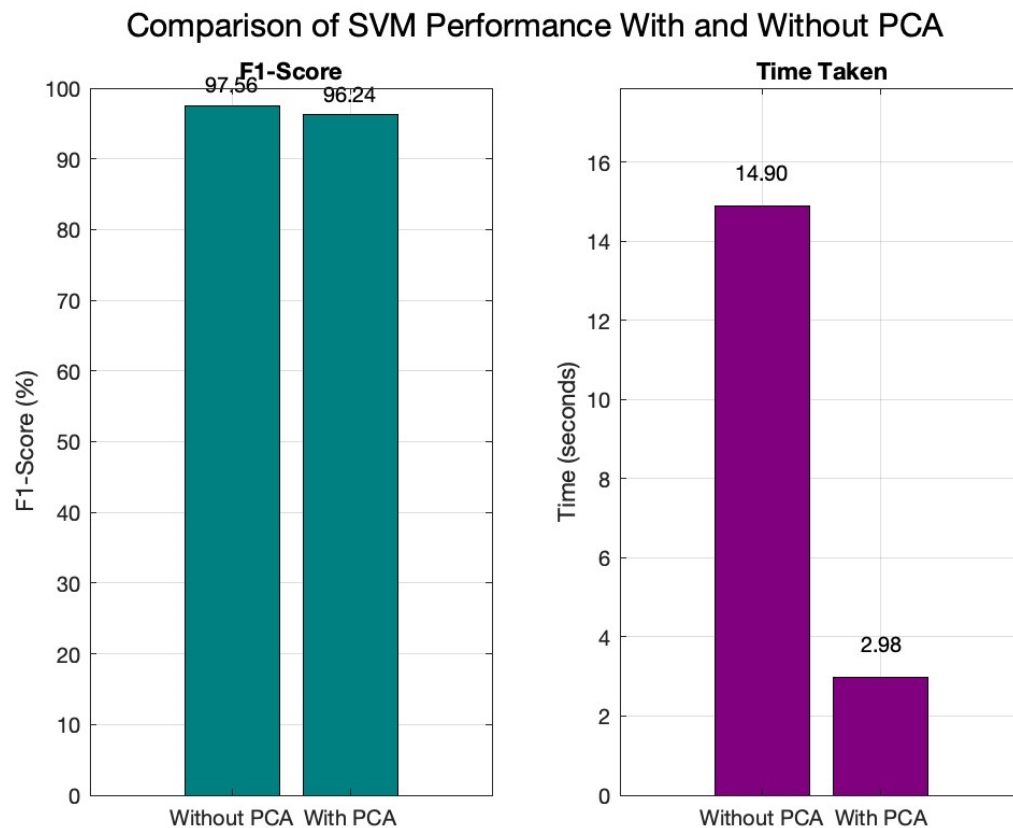


Figure 6.1

Figure 6.1 shows a modest decrease in F1 score when PCA is applied, accompanied by a sharp decrease in time taken. This is to be expected as PCA determines the most important features for classification – this usually results in a reduction of noise with an increase in computational efficiency by reducing the effect of the curse of dimensionality.

Due to the circumstances throughout the development of the detector, with a relatively small dataset and access to modern hardware which is capable of quickly training the SVM model, it was decided not to implement PCA, and to optimise for the highest possible F1 score, at the cost of computational efficiency.

7. Complete SVM Classifier

After the extensive tests and tuning detailed, the final machine learning model for the detector was developed. This was an SVM model using a homogenous polynomial kernel at 4 degrees. It is evaluated using a cross-validated F1 score.

The images the model is trained and tested on do not use preprocessing or PCA, as these were not found to have a positive effect on the cross-validated F1 score.

The images are processed with a Histogram of Gradients feature extractor before being used for training and testing.

The dataset is sampled at a rate of 1/1 – that is, every image in the dataset is used for both training and testing through cross-validation.

Before being used in the model, the images are augmented by rotating and flipping them, and these altered images are used to artificially inflate the dataset, as this was found to have a very positive effect on the F1 score of the model.

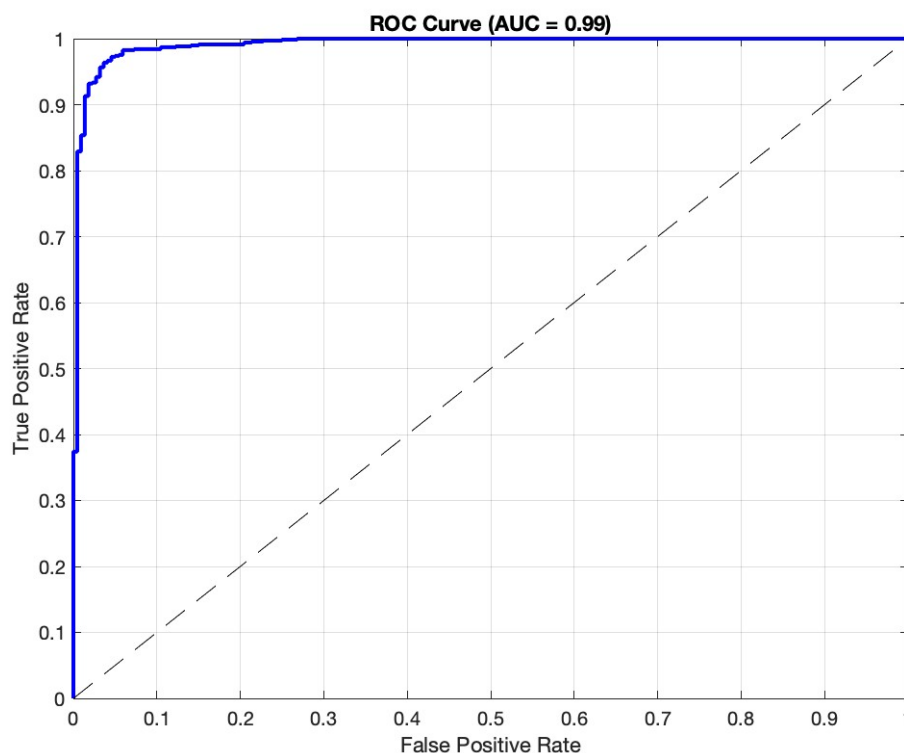


Figure 7.1

Figure 7.1 shows the ROC curve of the finished machine learning classifier.

Cross-Validation Accuracy: 96.26%

Confusion Matrix:

197	23
11	679

Precision: 96.72%

Recall: 98.41%

F1-Score: 97.56%

Figure 7.2

Figure 7.2 shows the confusion matrix, (mean) cross-validated accuracy, precision, recall, and F1 score. The model has very few type I errors or false positives, indicated by the high precision of 96.72%, and even better performance against type 2 errors or false negatives indicated by the recall of 98.41%. These values, as well as the F1 score of 97.56%, indicate the model is very robust and reliable.

There is an Area Under the Curve of 0.99, suggesting very high performance at classifying whether a given image has a face in it or not.

8. Sliding Window

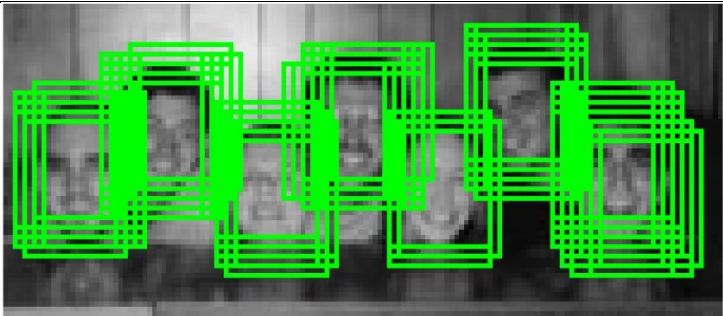
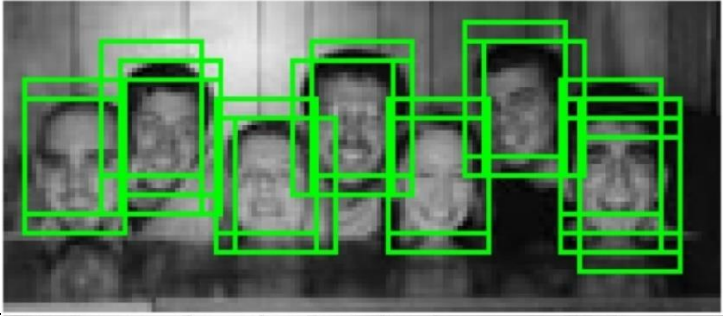
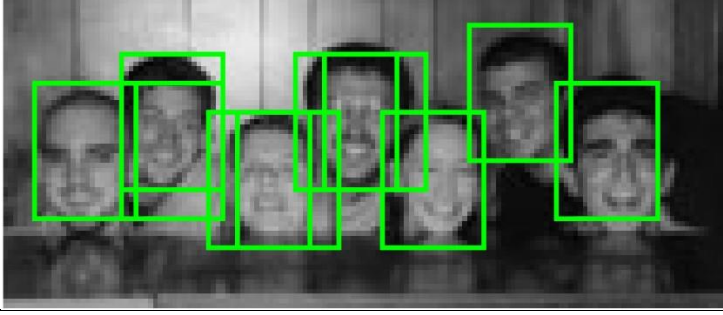
Running the Code

To run this code, simply run `sliding_window_svm` in the terminal, and the script will read in the datasets, train the SVM model, apply the model against each from of a sliding window at various scales on the test images, apply NMS, and output a highlighted version of each image along with the prediction values of each highlight. Ensure that the SVM-KM toolkit is added to the path and that the Computer Vision Toolbox add-on is installed on MATLAB.

8.1 Step Size

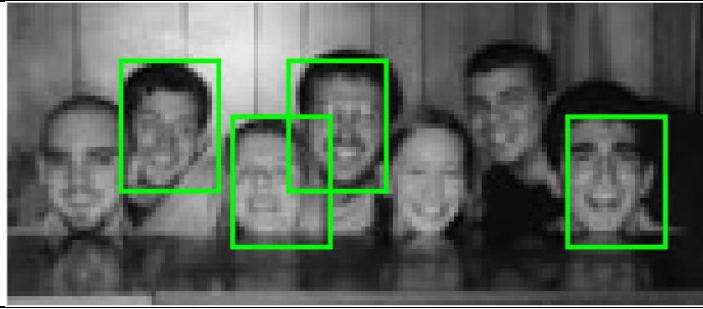
A lower step size means a faster processing speed, and less chance of overlapping face detections, however it also means there is a less chance of the sliding window to fit well on a given face, meaning we aren't giving the best chance for the model to detect the face. As long as NMS is implemented and configured will, a lower step size has no negative effects.

Here is an example of different step sizes (with no NMS or scaling applied)

Step Size	Output
2 Processing Time: 0.14	
4 Processing Time: 0.13	
6 Processing Time: 0.06	

12

Processing Time: 0.03



We found that a step size of 4 provides an optimal balance of processing speed and accuracy without NMS.

8.2 Non-Maximum Suppression (NMS)

We applied NMS to get past the issue with overlapping highlights on the same face, here are some example outputs with different IoU thresholds applied (without scaling):

For metrics we are using Number of False Negatives (Faces Missed),

Average Precision:

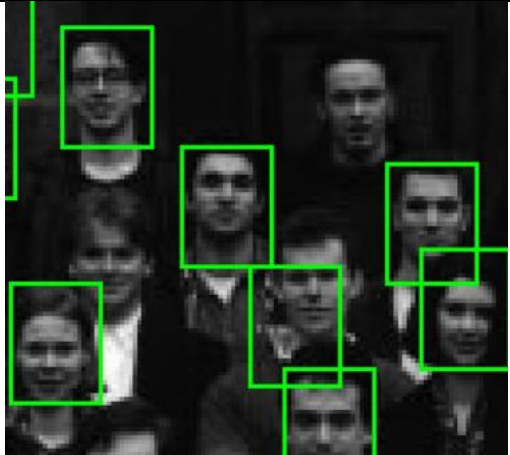
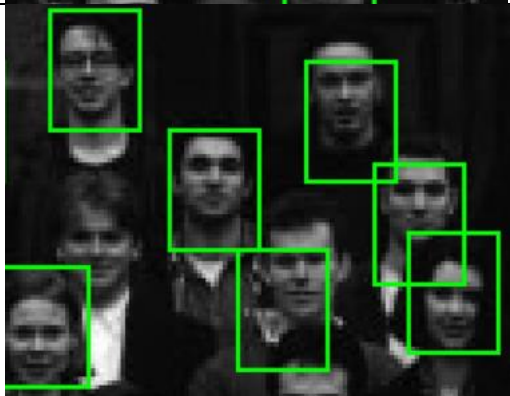
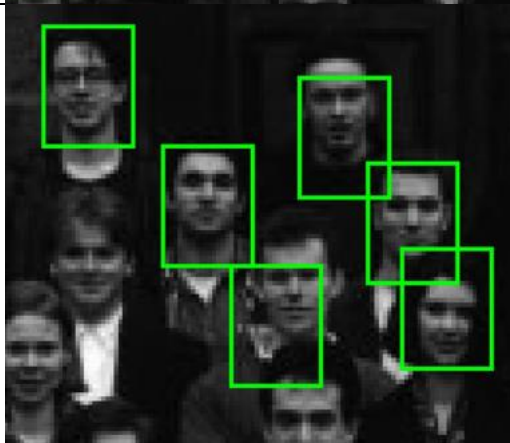
where precision is $P = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$

And Redundant Box Ratio - RBR:

$$RBR = \frac{\text{Total Detected Faces} - \text{False Positives} - (\text{Ground Truth Positives} - \text{False Negatives})}{\text{Total Detected Faces}}$$

Which is essentially the chance of a shown bounding box being redundant.

IoU Threshold	Output
1 (no NMS) Average Precision: 0.93 RBR: 0.72 False Negatives: 10	

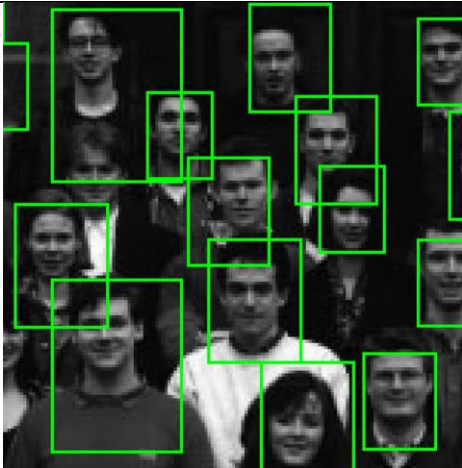
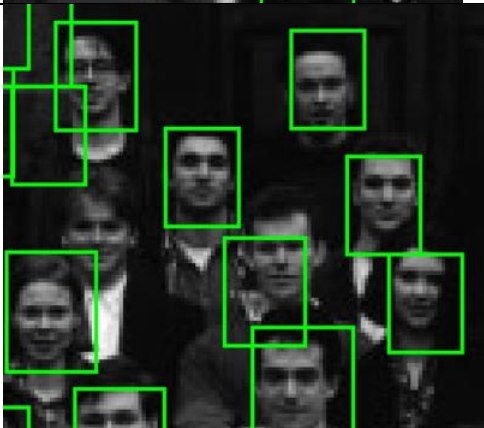
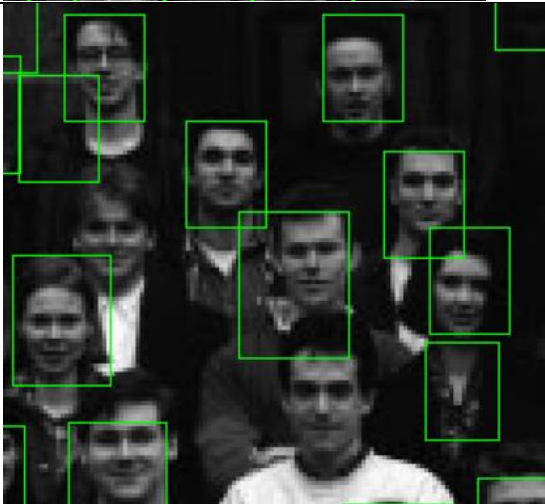
0.5 Average Precision: 0.862 RBR: 0.14 False Negatives: 8	
0.2 Average Precision: 0.890 RBR: 0.00 False Negatives: 10	
0.1 Average Precision: 0.895 RBR: 0.00 False Negatives: 13	

We found that a threshold of 0.2 is best, as it allows us to get rid of all the overlapping boxes with a RBR of 0, while not having any faces that were previously highlighted lost during the process. This is shown by its lower number of False Negatives (10) when compared to threshold of 0.1's (13).

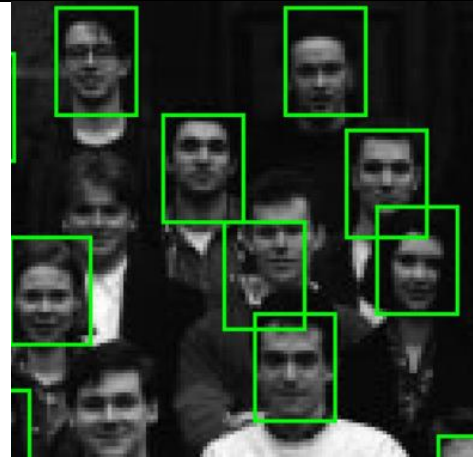
8.3 Scales

Applying the sliding window at different scales allows us to give the SVM classification model the best chance of detecting a face, as some faces in the image may be different sizes. The draw back of this is more overlapping selections (although this should be dealt with using NMS), and taking a lot more time, as you are doing the whole detection process an additional time for each scale you add.

Here is an example of the different selections when using scales. (The scale is applied to the image, so a small scale value corresponds to larger boxes)

Scales	Output
[1, 0.9, 0.8, 0.7, 0.5]	
[1.1, 1, 0.9, 0.8]	
[1.2, 1.1, 1, 0.9, 0.8]	

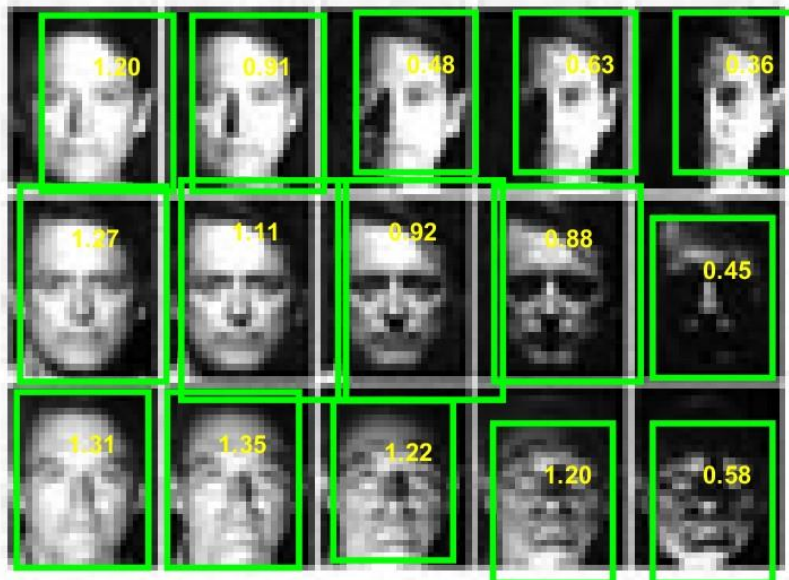
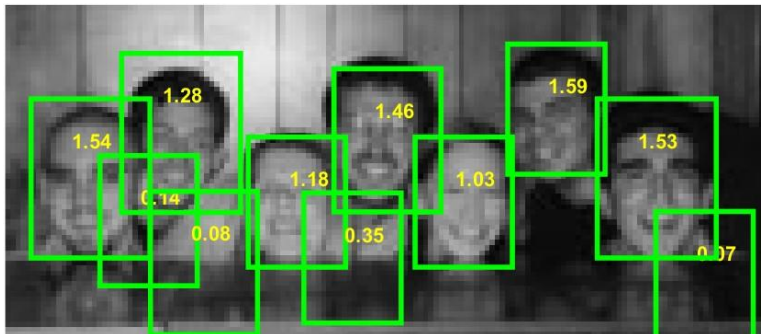
[1]



Using the scales [1.1, 1, 0.9, 0.8] seems to be optimal, as it allows the model to have a better chance at finding a face, while not confusing the model with false positives on over scaled inputs.

8.4 SVM Threshold Optimisation

When testing the script, I decided to label the image to get a better idea of how the model was performing on the test images. For example (using classification threshold of 0):



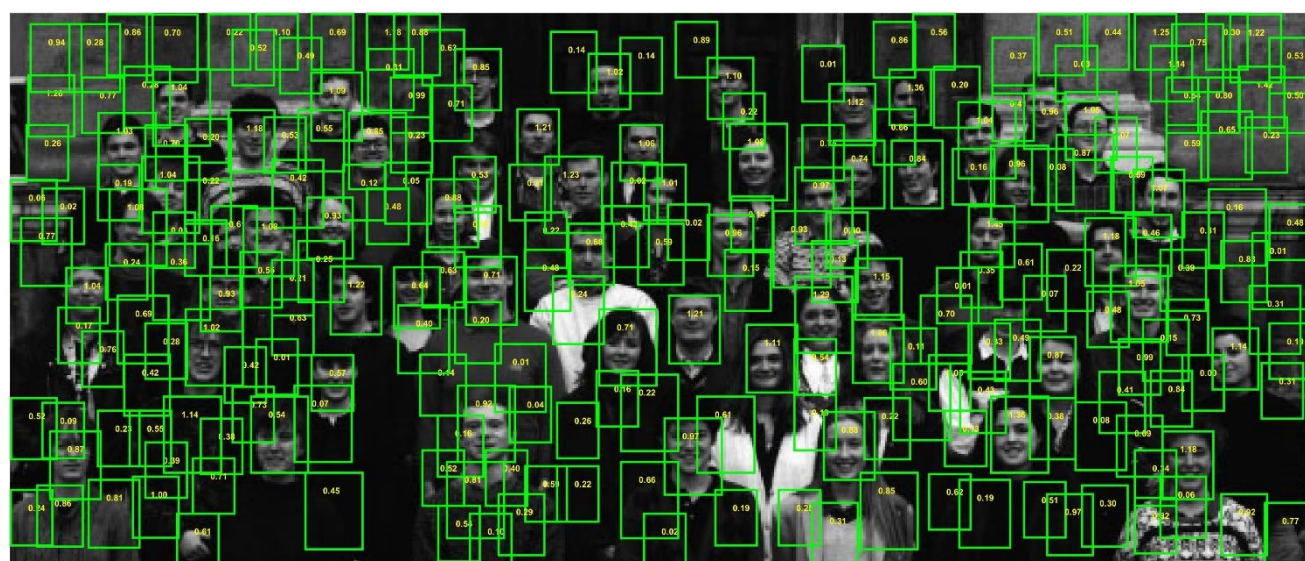
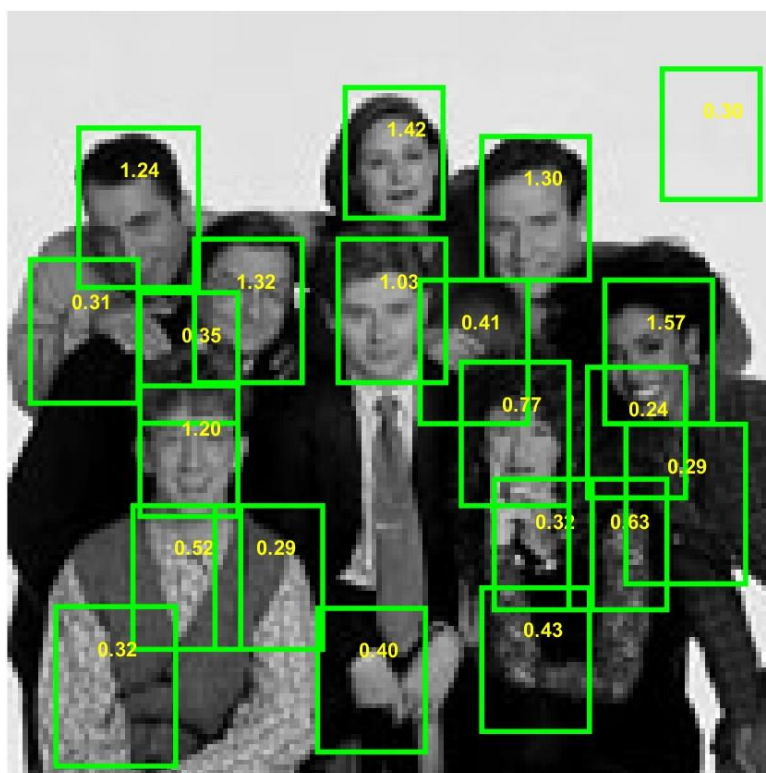
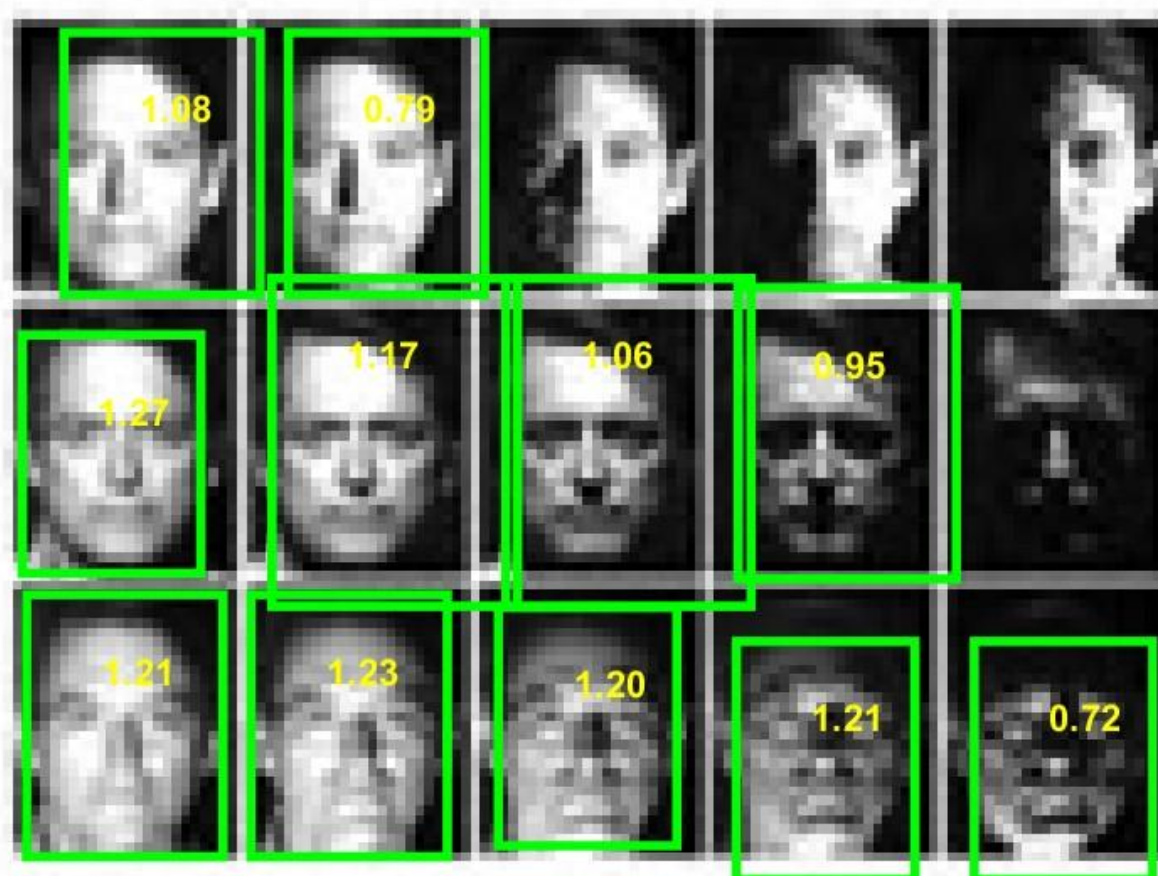
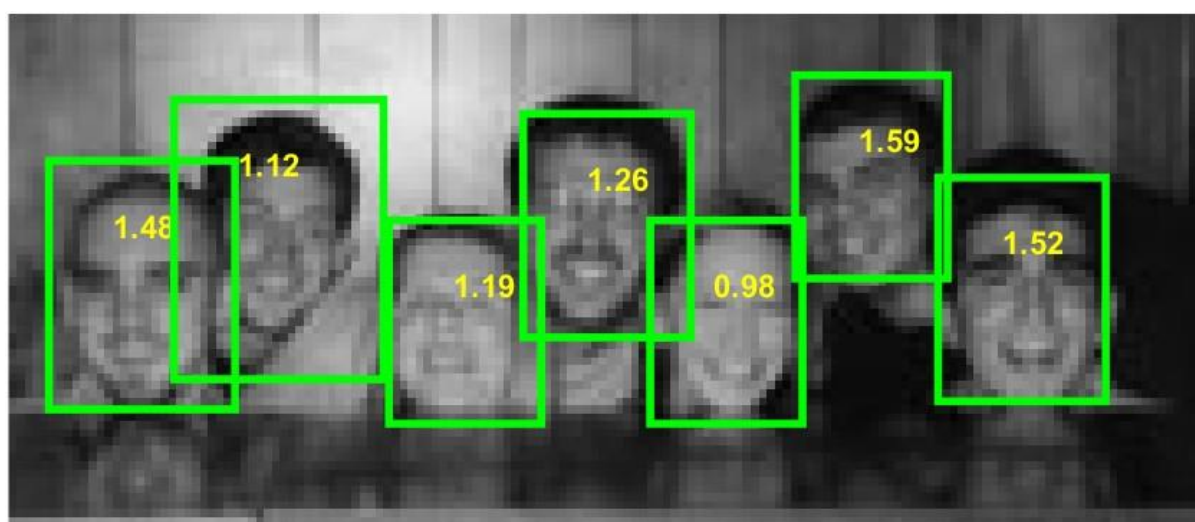


Image	Total Faces	Faces detected	Faces correctly detected	Faces Falsely detected	F1 Score
1	7	11	7	4	0.77
2	15	15	15	0	1
3	8	20	8	12	0.57
4	57	235	57	178	0.39

Average F1 Score = 0.68

With the exception of the second image, these outputs are unacceptable, but the labels allowed us to make an educated decision on what a good threshold would be, here is the outputs when using a threshold of 0.7:

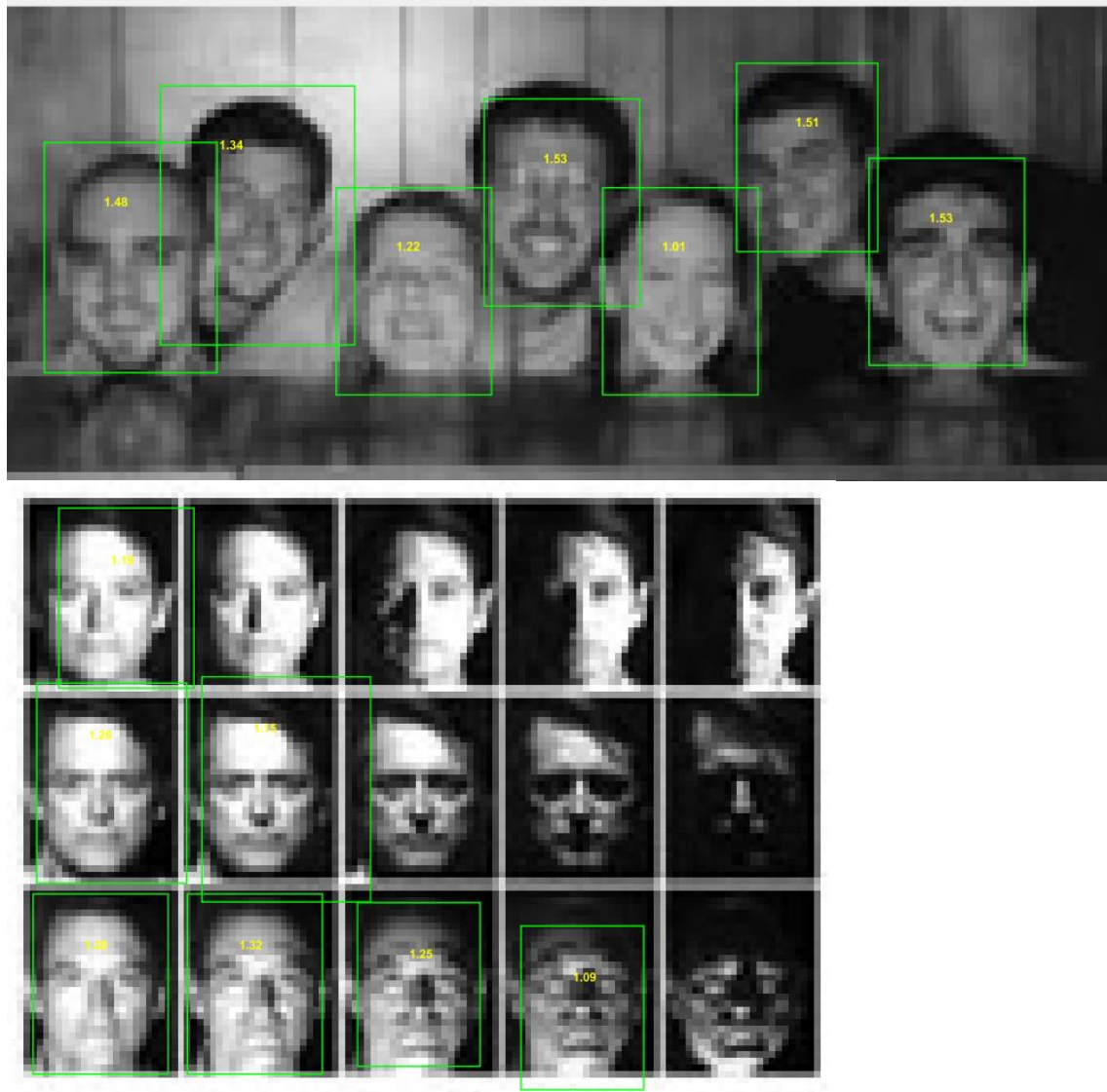


Although still not perfect, and with sacrificing accuracy when running on the second image, overall the accuracy is very good

Image	Total Faces	Faces detected	Faces correctly detected	Faces Falsely detected	F1 Score
1	7	7	7	0	1
2	15	10	10	0	0.8
3	8	8	8	0	1
4	57	93	52	41	0.69

Average F1 Score = 0.87

The outputs of 0.9:



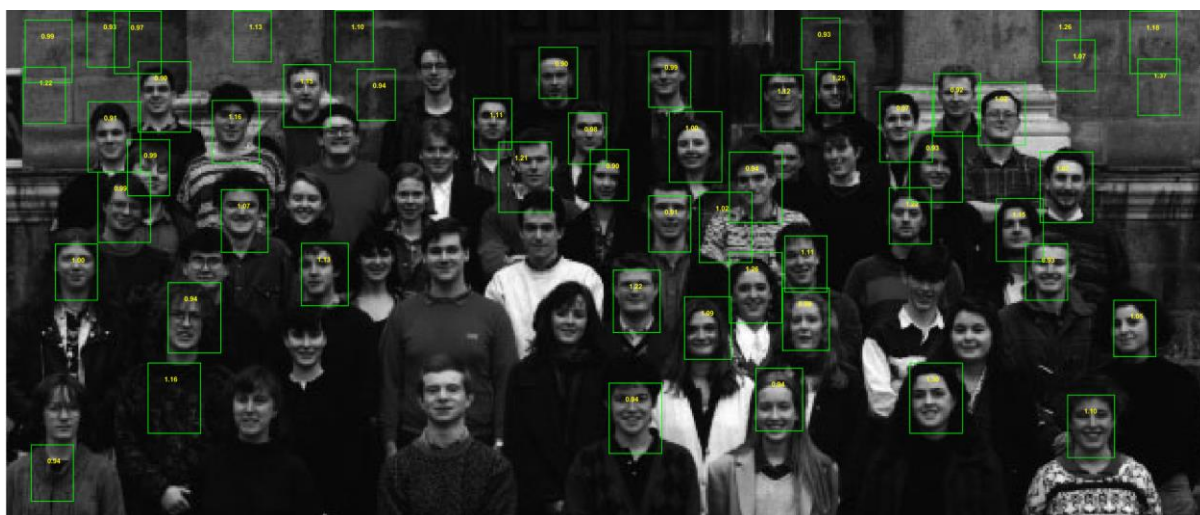
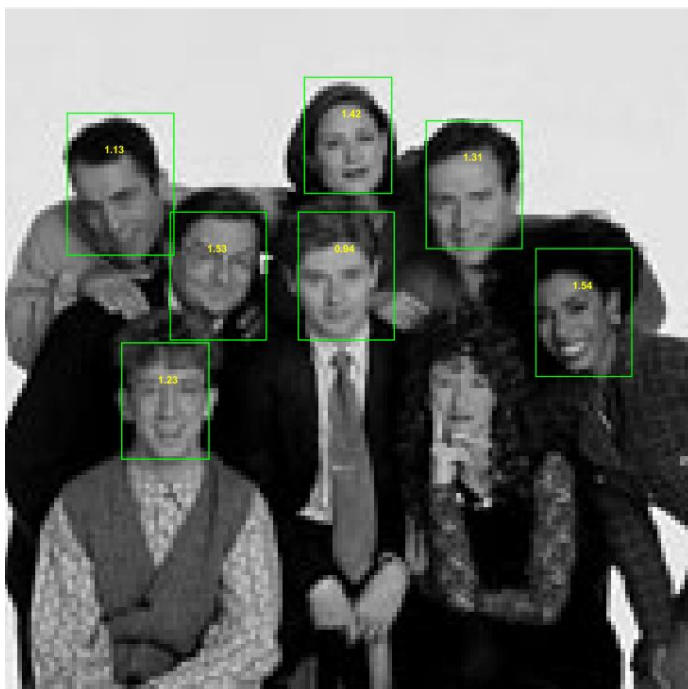


Image	Total Faces	Faces detected	Faces correctly detected	Faces Falsely detected	F1 Score
1	7	7	7	0	1
2	15	7	7	0	0.63
3	8	7	7	0	0.93
4	57	54	39	15	0.70

Average F1 Score: 0.81

The outputs of 0.5:

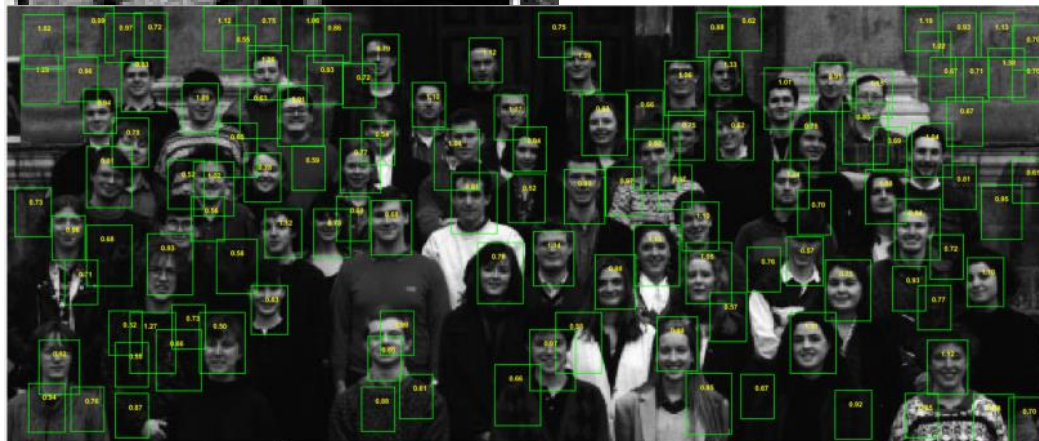
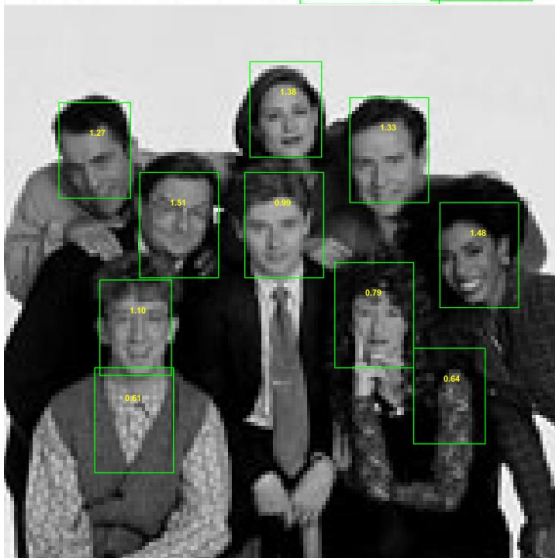
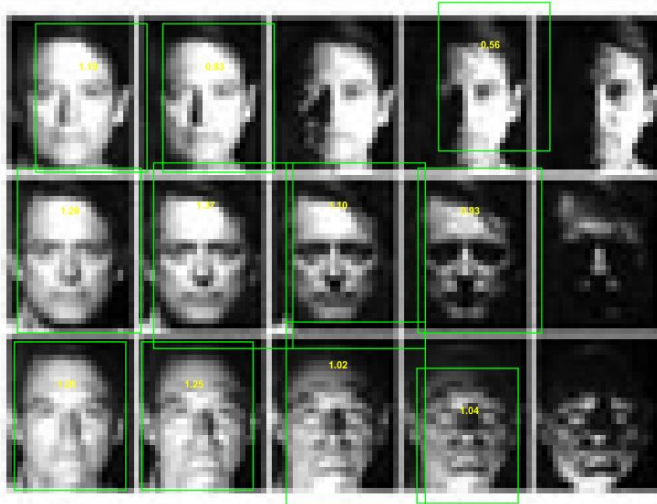
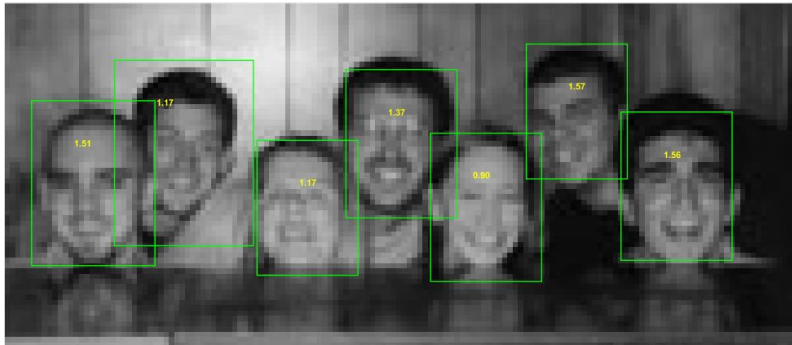
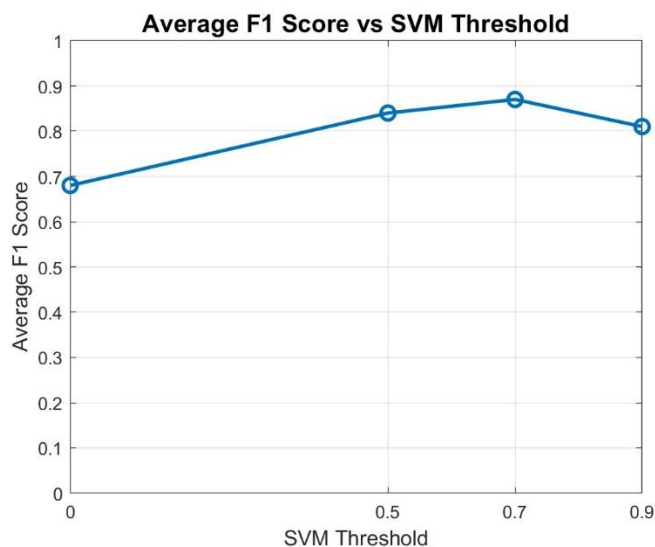


Image	Total Faces	Faces detected	Faces correctly detected	Faces Falsely detected	F1 Score
1	7	7	7	0	1
2	15	11	11	0	0.84
3	8	10	8	2	0.89
4	57	126	56	70	0.61

Average F1 Score: 0.84



I think the main limiting factor to the accuracy of this script, is the small dataset, and the variance/quality of that data, for example, the selection with the highest confidence value for image 4, being 1.30, isn't even a face, but part of a relatively uniform background. This is very bizarre, and I think having a larger dataset of training images with a wider range of the types of images being used would go far in solving this problem, this should hopefully allow us to reduce the prediction threshold, meaning we won't need to miss any correctly identified faces when attempting to not select false positives, this is evident in image 2, where the SVM model can clearly identify each face (although with lesser confidence as the light angle is more extreme on the faces), but because of the now higher prediction threshold, we are missing them. This may also be mitigated by having more faces in the training set with harsher light angles. To conclude, choosing a threshold of 0.7 will be more accurate than a threshold of 0, 0.5 or 0.9 as shown by its higher average F1 score across the images (0.87 compared to 0.68, 0.84 or 0.81).

9.0 Final Results

Our final parameter values for the sliding window script are as follows:

Sample rate	1
kFolds	6
Step size	4
Scales	[1.1, 1, 0.9, 0.8]
IoU Threshold	0.2
SVM Threshold	0.7

Running the script with these parameters produces the following results:

Image	Total Faces	Faces detected	Faces correctly detected	Faces Falsely detected	F1 Score
1	7	7	7	0	1
2	15	11	11	0	0.83
3	8	12	8	4	0.80
4	57	115	56	59	0.65

Average F1 Score: 0.82



